

PRISM-I Solution Options Matrix

MediSched AI

PRISM Phase: I – Intentional Solutioning

SmartX Technologies

Date: April 2026

Version: 1.0

PRISM-I Solution Options Matrix

SmartX Technologies

PRISM Phase: I – Intentional Solutioning

Purpose: Explore multiple viable solution approaches and deliberately select the best option based on tradeoffs, risks, and alignment with business and non-functional requirements.

1. Problem Recap

Problem Statement:

Diagnostic centers operate on manual, phone-based scheduling workflows with no centralized patient registry, no structured doctor availability management, and no call documentation. This leads to 15–30% patient no-show rates, 2–4 hours/day of staff time wasted on outbound calls, frequent double-bookings, and zero operational analytics. (Ref: PRISM-P Problem Brief)

Key Success Metrics (from PRISM-P):

- 70% reduction in manual outbound calling effort.
- 40% reduction in patient no-show rates.
- >85% appointment slot utilization.
- >50% AI call-to-booking conversion rate.
- >90% call transcription coverage.
- >70% doctor self-service slot adoption within 2 weeks.

Key Requirements (from PRISM-R):

- 50 functional requirements across 9 modules (Patient, Doctor, Slots, Calendar, Calling, Transcription, Reminders, Analytics, RBAC).
- 12 edge cases, 17 NFRs (performance, security, availability, scalability, observability, compliance).
- 4 user roles: Admin, Staff, Doctor, Read-Only. Plus 3 system actors: AI Calling Engine, Reminder Service, Transcription Service.
- 4 external integrations: Voice/Telephony (Twilio), STT (Whisper/Google), SMS Gateway, Email Service.

2. Constraints & Non-Negotiables

Time / Budget:

- Total project: 8 weeks. Functional prototype: 4 weeks.
- Open-source and free-tier services for prototype. Minimal cloud infrastructure spend.

- Single developer or small team (2–3 engineers) assumed for the prototype phase.

Technology (from Project Documentation — Indicative Stack):

The project documentation identifies the following technology options. The solution options in this document evaluate three distinct combinations drawn from this pool.

- Backend options: Python (FastAPI / Django) or Node.js (Express)
- Frontend options: React.js or Next.js
- Database options: PostgreSQL or MongoDB
- Voice/Telephony: Twilio or equivalent Voice API
- STT: OpenAI Whisper or Google Speech-to-Text
- Real-time: WebSockets + Redis
- Background processing: Celery (Python) or BullMQ (Node.js)

Compliance / Security:

- Encryption at rest (AES-256) and in transit (TLS 1.2+) — mandatory from Day 1.
- Role-based access control enforced on every API request (4 roles).
- Audit logging on all patient data and call recording access.
- Automated voice disclaimer on every AI outbound call.

Backward Compatibility:

- No existing system to maintain backward compatibility with — this is a greenfield build.
- The system must operate standalone with no EHR/EMR integration.

3. Evaluation Criteria

Each solution option will be evaluated against the following criteria, weighted by priority for MediSched AI's 8-week timeline and prototype-first approach.

Criterion	Weight	Description
Functional Fit	High	How completely does the option satisfy all 50 FRs from PRISM-R? Any gaps?
NFR Alignment	High	Does the option meet performance (2s load, 1s search), security (RBAC, encryption), real-time (2s propagation), and scalability (20 users, 10K patients) requirements?
Complexity	High	How complex is the architecture to build, debug,

		and maintain? Is it appropriate for a 2–3 person team in 8 weeks?
Time to Deliver	Critical	Can the option deliver a functional prototype in 4 weeks and full integration in 8 weeks?
Cost	Medium	Infrastructure, third-party service costs, and operational overhead during prototype and beyond.
Operational Risk	Medium	Deployment complexity, single points of failure, debugging difficulty, team skill requirements.

4. Solution Options Overview

Three genuinely different solution approaches have been evaluated, each using a distinct technology combination from the project documentation’s indicative stack. This ensures tradeoffs are evaluated across fundamentally different ecosystems, not just architectural variations of the same stack.

Option	Name	Summary
Option 1	Node.js (Express) + Next.js + MongoDB	Full JavaScript/TypeScript stack. Express.js REST API, Next.js frontend with SSR, MongoDB for flexible schema, BullMQ for background jobs. Unified language across frontend and backend.
Option 2	Django REST Framework + React.js + PostgreSQL (Recommended)	Python Django DRF backend with modular apps, standalone React.js SPA frontend, PostgreSQL for relational integrity, Celery workers for calling/transcription/reminders. Django Channels for real-time.
Option 3	Python FastAPI + Next.js + PostgreSQL	Async Python FastAPI backend, Next.js frontend with SSR/SSG, PostgreSQL via SQLAlchemy, ARQ/Celery for background tasks. High-performance async-first architecture.

5. Option 1 – Node.js (Express) + Next.js + MongoDB

Architecture Summary:

A full JavaScript/TypeScript stack using Node.js with Express.js as the REST API backend and Next.js as the frontend framework with server-side rendering. MongoDB serves as the primary database, offering flexible document schemas. BullMQ (Redis-backed) handles background jobs for calling, transcription, and reminders. Socket.io provides real-time WebSocket updates. A unified language (TypeScript) across frontend and backend.

Key Components:

- **Frontend (Next.js):** Next.js (React-based) with SSR/SSG for SEO-friendly pages and fast initial load. App Router with layouts for dashboard views.
- **Backend (Express.js):** Express.js REST API with TypeScript. Route handlers organized by domain (patients, doctors, slots, calls). Middleware-based auth and RBAC.
- **Database (MongoDB):** MongoDB via Mongoose ODM. Document-based schemas for patients, doctors, slots, call logs. Flexible schema allows rapid iteration.
- **Real-Time (Socket.io):** Socket.io for bidirectional WebSocket communication. Calendar updates broadcast to all connected clients on slot changes.
- **Background Jobs (BullMQ + Redis):** BullMQ with Redis. Job queues for call orchestration (with delayed retry for 2-hour escalation), transcription processing, and reminder dispatch.
- **Voice/Telephony (Twilio):** Twilio Node.js SDK for outbound calls. TwiML served via Express routes for IVR callback handling.
- **STT:** Google Speech-to-Text Node.js client library or OpenAI Whisper API called from BullMQ worker post-call.
- **Auth/RBAC (Passport.js + JWT):** Passport.js with JWT strategy. Role-based middleware (isAdmin, isStaff, isDoctor, isReadOnly) applied per route.
- **Deployment:** Single Docker Compose with 4 services: next-app, express-api, mongodb, redis. Deployable on Vercel (frontend) + Railway (backend).

How Requirements Are Met:

- **FR-PM:** Mongoose Patient model with schema validation (required: name, phone, DOB, gender). Express route with search (regex on name/phone). Duplicate phone check via unique index.
- **FR-DM/DS:** Mongoose Doctor and DoctorSlot models. Slot auto-generation function creates slot documents. Overlap detection via MongoDB query on (doctor, date, startTime). Atomic findOneAndUpdate for concurrent booking.
- **FR-CM:** Next.js page with FullCalendar component. REST API fetches slots. Socket.io receives live updates. Color-coded slot statuses in React.
- **FR-AC:** BullMQ job orchestrates 3-attempt escalation. Twilio SDK initiates call. TwiML webhooks hit Express endpoints. Delayed job (2-hour) for retry. Manual callback record on Attempt 3 failure.

- FR-CD: BullMQ worker downloads recording, sends to STT API, stores transcription in MongoDB document linked to call log and appointment.
- FR-RM: BullMQ repeatable job checks upcoming appointments. Worker sends SMS (Twilio) and email (Nodemailer/SendGrid). Delivery status logged.
- FR-AN: Express endpoint with MongoDB aggregation pipeline (\$group, \$match, \$project). CSV export via json2csv stream.
- FR-UA: Passport.js JWT middleware + role-checking middleware on every route group. Returns HTTP 403 for unauthorized access.

6. Option 1 – Pros / Cons / Risks

Pros:

- Unified TypeScript language across frontend and backend — zero context switching, shared type definitions, single toolchain (npm/pnpm).
- Next.js provides SSR, SSG, and API routes out of the box — fast initial page loads and SEO potential.
- MongoDB's flexible schema allows rapid iteration on data models without migrations during early prototyping.
- BullMQ is a mature, Redis-backed job queue for Node.js — supports delayed jobs (2-hour call retry), retries, and a built-in dashboard (Bull Board).
- Socket.io is the most widely-used WebSocket library with automatic reconnection, room management, and broad browser support.
- Largest developer ecosystem — npm packages for virtually every integration (Twilio, Google STT, Mongoose, Nodemailer).
- Low-cost deployment: Vercel free tier for Next.js + Railway for Express + MongoDB Atlas free tier.

Cons:

- MongoDB lacks referential integrity enforcement — no foreign key constraints between patients, doctors, slots, and appointments. Data consistency must be enforced at the application layer, increasing bug risk for a scheduling system where relational correctness is critical.
- No built-in admin panel — requires building a custom admin UI or integrating a third-party tool (AdminBro/AdminJS), adding development time that Django provides for free.
- Mongoose ODM is less mature than Django ORM for complex queries — aggregation pipelines are powerful but verbose and harder to debug than Django's queryset API.
- Node.js Whisper integration is less mature than Python's. The Python Whisper library supports local model execution; Node.js requires API-only access or a Python subprocess.

- Express.js provides minimal structure — no built-in serialization, validation, or permission framework. These must be assembled manually (Joi/Zod for validation, custom middleware for RBAC), increasing boilerplate.
- MongoDB's document model is a poor fit for the highly relational MediSched data model (patient → appointment → slot → doctor) — queries involving joins require \$lookup pipelines or denormalization, both adding complexity.
- No equivalent of Django's built-in migration framework — MongoDB schema changes are schemaless, which is fast initially but creates data consistency risks over time.

Risks and Unknowns:

- Data integrity risk (Critical) — without database-level foreign key constraints, orphaned references (e.g., a booking pointing to a deleted slot) can silently corrupt data. Mitigation: application-level validation, but this is fragile under concurrent access.
- Concurrent slot booking race condition — MongoDB's findOneAndUpdate with version checking can handle this, but it's less robust than PostgreSQL's SELECT FOR UPDATE with row-level locking.
- AI/ML ecosystem gap — Python is the dominant language for AI/voice processing. Using Node.js for Twilio + STT means fewer libraries, less community support, and potential need for a Python sidecar for Whisper local model.

7. Option 2 – Django REST API + React SPA + Celery Workers

Architecture Summary:

A Django REST Framework backend serves as the core API layer, organized into modular Django apps (patients, doctors, scheduling, calling, transcriptions, analytics, users). A standalone React.js SPA (built with Vite) communicates with the backend via REST APIs. Django Channels provides WebSocket support for real-time calendar updates via Redis pub/sub. Dedicated Celery workers (using Redis as broker) handle all background processing: AI call orchestration, transcription, and reminder dispatch. All components share a single PostgreSQL database.

Key Components:

- **Frontend (React.js):** React.js SPA (Vite build) for all UI: calendar views, patient management, doctor slot management, call logs, analytics dashboard. Communicates with Django backend via Axios/fetch over REST. WebSocket connection for real-time calendar updates.
- **Backend API (Django DRF):** Django 5.x + Django REST Framework. Modular Django apps: patients, doctors, scheduling, calling, transcriptions, analytics, users. Each app has its own models, serializers, viewsets, permissions, and service layer.
- **Database (PostgreSQL):** PostgreSQL via Django ORM. Alembic-free — Django's built-in migration framework (makemigrations / migrate). Database constraints enforce referential integrity (ForeignKey, unique_together).

- **Real-Time (Django Channels + Redis):** Django Channels with Redis channel layer. ASGI server (Daphne or Uvicorn) handles WebSocket connections. Backend publishes slot-change events to Redis; Channels broadcasts to all connected React clients.
- **Background Workers (Celery + Redis):** Celery workers with Redis as message broker. Three dedicated task queues:
 - (a) `calling_queue` — orchestrates outbound AI calls, 3-attempt escalation with precise scheduling (Celery ETA/countdown for 2-hour retry).
 - (b) `transcription_queue` — receives call recording URLs post-call, downloads audio, runs Whisper API, stores transcription, links to appointment.
 - (c) `reminder_queue` — Celery Beat periodic task checks upcoming appointments (24-hour window). Sends SMS (Twilio) and email (Django `send_mail` / SendGrid).
- **Voice/Telephony (Twilio):** Twilio Python SDK for outbound calls, TwiML generation, and webhook handling. Django URL endpoints serve as TwiML callback targets.
- **STT (Whisper):** OpenAI Whisper API called from Celery transcription worker post-call. Option to switch to local Whisper model on GPU server for production cost optimization.
- **Auth/RBAC (JWT + DRF Permissions):** Django REST Framework Simple JWT. JWT access + refresh tokens. Role claim embedded in token payload. Custom DRF permission classes (`IsAdmin`, `IsStaff`, `IsDoctor`, `IsReadOnly`) enforce RBAC per viewset action.
- **Deployment (Docker Compose):** Docker Compose for local development and deployment. 5 services: `react-frontend`, `django-api`, `celery-worker`, `postgresql`, `redis`. Single `docker-compose.yml` manages the full stack. Deployable on Railway, Render, or any VPS.

How Requirements Are Met:

- **FR-PM (Patient Management):** Django "patients" app — Patient model with required/optional fields, DRF `ModelSerializer` with validation, `ModelViewSet` with search filter (name, phone). Duplicate phone detection via unique constraint + custom `validate_phone_number()`. Appointment history and call logs exposed via nested serializers on patient detail endpoint.
- **FR-DM/DS (Doctor & Slots):** Django "doctors" app — Doctor model + DoctorSlot model. Slot auto-generation: service function accepts (doctor, date, start_time, end_time, duration) and bulk-creates DoctorSlot objects. Overlap detection via queryset filter on (doctor, date, start_time) before creation. Optimistic locking via `select_for_update()` for concurrent booking.
- **FR-CM (Calendar):** React FullCalendar component fetches slot data from DRF endpoint (filterable by doctor, date range, status). WebSocket connection via Django Channels receives real-time slot-change events. Color-coding logic in React: Available=green, Booked=blue, Blocked=grey.
- **FR-AC (Calling):** Celery task `call_patient.delay(patient_id, attempt=1)` initiates outbound call via Twilio. TwiML webhooks hit Django endpoints for IVR response handling. On no-answer: Celery schedules retry with `call_patient.apply_async(eta=now+2hours, kwargs={attempt:`

2}). On Attempt 3: creates ManualCallbackTask record for staff. All attempts logged to CallLog model.

- FR-CD (Transcription): Celery task transcribe_call.delay(call_log_id) triggered on call completion. Downloads recording from Twilio, sends to Whisper API, stores transcription text in Transcription model linked to CallLog and Appointment. Staff can edit via DRF endpoint.
- FR-RM (Reminders): Celery Beat runs check_upcoming_appointments every hour. For appointments within 24-hour window, dispatches send_reminder.delay(appointment_id). Worker sends SMS (Twilio) and email (Django send_mail). ReminderLog model tracks delivery status.
- FR-AN (Analytics): DRF "analytics" app with aggregate endpoints using Django ORM (Count, Avg, F expressions). Filterable by date range, doctor, service type. CSV export via StreamingHttpResponse with csv.writer.
- FR-UA (RBAC): Custom DRF permission classes applied at viewset level. JWT token contains user role. Every API request validated against role permissions. Unauthorized access returns HTTP 403 with error message. Django admin restricted to superusers only.

8. Option 2 – Pros / Cons / Risks

Pros:

- Django REST Framework is the most battle-tested Python REST API framework — built-in serialization, validation, authentication, pagination, throttling, and browsable API for development.
- Django ORM with PostgreSQL provides the strongest relational data modeling in the Python ecosystem — migrations, constraints, transactions, and admin panel come free.
- Django admin panel gives instant visibility into all data (patients, doctors, slots, call logs) without building custom admin UI — invaluable for debugging and prototype demos.
- Celery is the industry-standard Python task queue — reliable scheduling (ETA, countdown), automatic retries with exponential backoff, dead-letter queues, and task monitoring via Flower.
- Celery workers isolate long-running tasks (AI calls, transcription, reminders) from the Django API process — no impact on API response times or WebSocket stability.
- Django Channels with Redis channel layer delivers reliable real-time WebSocket updates for the calendar — well-proven at the 20-concurrent-user scale required.
- React.js as a standalone SPA enables parallel frontend/backend development, independent deployment, and access to the full React ecosystem (FullCalendar, React Query, React Router).
- Docker Compose (5 services) provides reproducible local development that mirrors production exactly.

- Django's built-in security features (CSRF, XSS protection, SQL injection prevention, password hashing) reduce the security implementation burden.
- Achievable by 2–3 developers within the 8-week timeline. Prototype (scheduling + patient/doctor management) at Week 4. Full build by Week 8.

Cons:

- Two separate codebases (Django backend + React frontend) require coordination for API contracts, CORS configuration, and deployment.
- Celery + Redis adds operational complexity — Celery workers, Celery Beat scheduler, Redis broker, and Flower monitoring are additional processes to manage.
- Docker Compose with 5 services has a higher setup barrier than Option 1's single-process approach.
- Django's synchronous ORM requires django-channels and ASGI for async WebSocket support, adding configuration complexity.
- DRF's viewset/serializer pattern has a learning curve for developers new to Django.

Risks and Unknowns:

- Celery worker crash during call orchestration — Mitigation: Celery auto-retry with exponential backoff; Redis-backed task persistence ensures jobs survive worker restarts; Docker restart policies auto-recover crashed containers; idempotent task design.
- Django Channels WebSocket stability under 20 concurrent connections — Mitigation: Redis channel layer (not in-memory); load test at Week 3; Daphne/Uvicorn ASGI server is production-grade.
- Whisper API latency (5–15s per transcription) — Mitigation: transcription is fully asynchronous via Celery. Call logs show "Transcription in progress" until complete. No user-facing delay.
- Docker Compose environment drift across developers — Mitigation: .env.example template, docker-compose.yml pinned versions, CI validates Docker build on every push.

9. Option 3 – Python FastAPI + Next.js + PostgreSQL

Architecture Summary:

A high-performance async Python backend using FastAPI serves the REST API. Next.js provides the frontend with SSR capabilities. PostgreSQL via SQLAlchemy ORM stores all relational data. ARQ (async Redis queue) or Celery handles background tasks. FastAPI's native WebSocket support provides real-time calendar updates. The architecture prioritizes raw performance and async-first design.

Key Components:

- **Frontend (Next.js):** Next.js (React-based) with App Router. SSR for initial page load. Client-side React for interactive dashboard components.
- **Backend API (FastAPI):** FastAPI with async endpoint handlers. Pydantic models for request/response validation. Modular router structure (patients, doctors, scheduling, calling, analytics).
- **Database (PostgreSQL + SQLAlchemy):** PostgreSQL via SQLAlchemy 2.0 (async) + Alembic for migrations. Relationship-based models with foreign keys, unique constraints, and indexes.
- **Real-Time (FastAPI WebSocket + Redis):** FastAPI native WebSocket endpoints + Redis pub/sub for broadcasting slot-change events to all connected Next.js clients.
- **Background Workers (ARQ/Celery + Redis):** ARQ (lightweight async Redis queue) or Celery. Task queues for call orchestration, transcription, and reminders. ARQ is async-native and lighter than Celery.
- **Voice/Telephony (Twilio):** Twilio Python SDK for outbound calls. Webhook endpoints in FastAPI for TwiML IVR handling.
- **STT (Whisper):** OpenAI Whisper (local model via Python library or API). Python is the native ecosystem for Whisper — direct model loading and inference possible.
- **Auth/RBAC (JWT + FastAPI Depends):** python-jose for JWT. FastAPI Depends() for dependency-injected RBAC per endpoint.
- **Deployment (Docker Compose):** Docker Compose with 5 services: next-frontend, fastapi-api, arq-worker, postgres, redis.

How Requirements Are Met:

- **FR-PM:** SQLAlchemy Patient model. FastAPI patient router with Pydantic schemas for validation. Async CRUD operations. Duplicate phone check via unique constraint + database exception handling.
- **FR-DM/DS:** Doctor and DoctorSlot SQLAlchemy models. Slot auto-generation as async service function. Overlap detection via async SQLAlchemy query. Pessimistic locking via SELECT FOR UPDATE for concurrent booking.
- **FR-CM:** Next.js page with FullCalendar + WebSocket connection to FastAPI. REST API for slot data fetching. Live updates via WebSocket on slot status changes.
- **FR-AC:** ARQ/Celery task orchestrates 3-attempt escalation. Twilio SDK for calls. FastAPI webhook endpoints for IVR. Delayed task scheduling for 2-hour retry.
- **FR-CD:** Worker downloads recording, runs Whisper (local model or API), stores transcription in SQLAlchemy Transcription model linked to call log and appointment.
- **FR-RM:** Periodic worker task checks upcoming appointments. Sends SMS (Twilio) and email (aiosmtplib/SendGrid). Delivery status logged.
- **FR-AN:** FastAPI endpoint with SQLAlchemy aggregate queries (func.count, func.avg). StreamingResponse for CSV export.
- **FR-UA:** JWT with role claim. FastAPI Depends() injects current_user with role. Custom dependency functions (require_admin, require_staff, etc.) per router. Returns HTTP 403.

10. Option 3 – Pros / Cons / Risks

Pros:

- Highest raw performance — FastAPI is built on Starlette (ASGI) and is one of the fastest Python frameworks, with native async/await support throughout.
- Pydantic provides automatic request/response validation and serialization with type safety — fewer runtime errors, auto-generated OpenAPI docs.
- FastAPI auto-generates interactive Swagger UI and ReDoc API documentation — excellent for frontend integration and demo.
- Native WebSocket support in FastAPI — no need for a separate library like Django Channels.
- Python is the best ecosystem for AI/voice workloads — Whisper can run locally as a Python library (no API cost), Twilio SDK is mature, and data processing libraries (pandas, numpy) are available for analytics.
- SQLAlchemy 2.0 (async) with PostgreSQL provides strong relational modeling with foreign keys, transactions, and optimistic/pessimistic locking.
- ARQ (async Redis queue) is lighter and faster than Celery for async-first architectures.
- Next.js provides SSR for fast initial loads and a mature React ecosystem.

Cons:

- No built-in admin panel — Django's admin is a major productivity advantage that FastAPI lacks entirely. Building an admin interface requires custom development or third-party tools (SQLAdmin), adding 1–2 weeks to the timeline.
- SQLAlchemy + Alembic has a steeper learning curve than Django ORM — more verbose model definitions, manual migration generation, and less opinionated project structure.
- FastAPI provides no project structure convention — developers must design the folder layout, dependency injection pattern, and service layer from scratch. Django's app-based structure provides this for free.
- ARQ is less mature than Celery — smaller community, fewer monitoring tools (no Flower equivalent), less documentation. Celery can be used instead but negates the "lightweight" advantage.
- Two-language stack (Python + TypeScript for Next.js) with neither language having a "batteries-included" framework — more assembly required at every layer compared to Django DRF.
- FastAPI's ecosystem is younger — fewer battle-tested plugins for auth, permissions, pagination, and throttling compared to DRF.
- SQLAlchemy async mode has known rough edges with session management and lazy loading — requires careful scoping of database sessions per request.

Risks and Unknowns:

- Prototype timeline risk (High) — FastAPI’s lack of built-in admin, project structure, and DRF-equivalent tooling means more code must be written from scratch. Estimated 1–2 week overhead vs. Option 2 for equivalent functionality.
- SQLAlchemy async session management — improper session scoping can cause connection leaks or stale data. Mitigation: use contextvar-based session factory, but requires discipline.
- ARQ reliability under production load is less proven than Celery — Mitigation: fall back to Celery if ARQ shows instability, but this adds migration effort.
- No Django admin fallback for prototype demos — all admin functionality must be built as custom UI. Risk of incomplete admin features under time pressure.

11. Tradeoff Comparison Matrix

Each option is scored on a 5-point scale: 5 = Excellent, 4 = Good, 3 = Adequate, 2 = Weak, 1 = Poor.

Criterion (Weight)	Option 1: Express + Next.js + MongoDB	Option 2: Django DRF + React + PostgreSQL	Option 3: FastAPI + Next.js + PostgreSQL
Functional Fit (High)	3 — Meets FRs functionally but MongoDB lacks referential integrity for the relational data model. Application-level enforcement is fragile.	5 — Meets all 50 FRs. Django ORM + PostgreSQL excels for relational data. DRF provides built-in serialization, validation, and permissions.	4 — Meets all FRs. SQLAlchemy + PostgreSQL is strong. But no admin panel and more manual assembly required.
NFR Alignment (High)	3 — Performance OK. Security achievable but manual (Passport.js + custom middleware). MongoDB lacks ACID for multi-document transactions by default.	5 — All NFRs met. Django’s built-in security (CSRF, XSS, SQL injection). Celery + Channels for scalability and real-time. JWT + DRF permissions for RBAC.	5 — All NFRs met. FastAPI is high-performance async. SQLAlchemy provides ACID. Native WebSocket. JWT + Depends for RBAC.
Complexity (High)	3 — Simple single-language stack, but Express lacks structure. No admin panel. Manual validation, RBAC, serialization assembly.	4 — Moderate. Django backend + React + Celery + Docker. But Django’s batteries-included approach reduces custom code significantly.	3 — FastAPI is lightweight but requires manual project structure, admin UI, session management. SQLAlchemy async has rough edges.
Time to Deliver	4 — Fast prototyping	5 — Fastest to	3 — 1–2 weeks

(Critical)	with JS ecosystem. But no admin panel adds 1–2 weeks. MongoDB schema flexibility saves time early but costs later.	production-quality prototype. Django admin, DRF viewsets, built-in migrations, Celery — all reduce boilerplate. Prototype in 4 weeks.	slower than Option 2 due to manual admin UI, Alembic setup, project structure design. Prototype at risk of Week 5.
Cost (Medium)	4 — Low cost. Vercel + Railway + MongoDB Atlas free tiers. Google STT may have higher per-call cost than Whisper.	4 — Low cost. Docker on Railway/Render or VPS (\$5–10/month). Whisper API or local model.	4 — Low cost. Similar to Option 2. Local Whisper model possible (Python native).
Operational Risk (Medium)	3 — BullMQ is solid for jobs, but MongoDB data integrity risk and Express’s minimal structure increase debugging difficulty.	5 — Lowest operational risk. Celery is battle-tested. Django’s ORM prevents data corruption. Flower for task monitoring. Docker for consistency.	3 — ARQ is less proven than Celery. SQLAlchemy async session management requires discipline. No Flower equivalent for ARQ.

Weighted Score Summary:

Option	Score	Assessment
Option 1: Express + Next.js + MongoDB	20 / 30	Unified language is appealing but MongoDB’s lack of relational integrity and Express’s minimal structure are critical weaknesses for a scheduling system.
Option 2: Django DRF + React.js + PostgreSQL	28 / 30	Best overall. Batteries-included framework, strongest ORM, free admin panel, battle-tested Celery, built-in security. Production-ready in 8 weeks.
Option 3: FastAPI + Next.js + PostgreSQL	22 / 30	High performance and strong Python AI ecosystem, but no admin panel, steeper learning curve, and slower prototype delivery.

12. Recommended Option

Recommendation: Option 2 – Django REST Framework + React.js + PostgreSQL + Celery

Why this option?

- Highest overall score (28/30) across all evaluation criteria — no critical weaknesses in any category.
- Meets all 50 functional requirements from PRISM-R without compromise.
- Django REST Framework is the most batteries-included API toolkit evaluated — serialization, validation, browsable API, throttling, pagination, permissions, and authentication all built-in. Compared to Express.js (Option 1) and FastAPI (Option 3), DRF requires the least custom code to achieve equivalent functionality.
- Django ORM + PostgreSQL provides the strongest relational data modeling — built-in migrations, foreign key constraints, transactions, and query optimization. This is critical for MediSched’s relational data model (patients → appointments → slots → doctors). MongoDB (Option 1) cannot enforce these relationships at the database level.
- Django admin panel is a decisive advantage — instant back-office interface for data inspection, user management, and prototype demos with zero additional development. Neither Express (Option 1) nor FastAPI (Option 3) offers this.
- Celery with Redis is the industry-standard Python task queue — precise scheduling (ETA for 2-hour call retries), automatic retries with backoff, dead-letter queues, and Flower monitoring. More mature and better monitored than BullMQ (Option 1) and ARQ (Option 3).
- Python is the strongest ecosystem for AI/voice workloads — native Whisper support, mature Twilio SDK. While Option 3 (FastAPI) shares this advantage, Option 1 (Node.js) does not.
- Django Channels + Redis delivers reliable WebSocket-based real-time calendar updates. While FastAPI (Option 3) has native WebSocket, Django Channels is more battle-tested for production use with Redis channel layers.
- React.js as a standalone SPA allows parallel frontend/backend development with the full React ecosystem (FullCalendar, React Query, React Hook Form).
- Django’s built-in security (CSRF, XSS, SQL injection protection, password hashing) provides the strongest security baseline of all three options.
- Clear upgrade path: modular Django apps can be extracted into microservices when scale demands, without re-architecture.

What tradeoffs are being accepted?

- Two-language stack (Python + JavaScript) — accepted because Python’s AI/voice ecosystem and Django’s productivity advantages far outweigh the unified-language benefit of Option 1 (Node.js). React is standard frontend regardless.

- Celery + Redis adds operational components vs. simpler alternatives — accepted because reliable, monitorable background processing is non-negotiable for AI call orchestration.
- Django's synchronous ORM requires ASGI + Channels for WebSocket — accepted because Django Channels is mature and the configuration overhead is minimal compared to FastAPI's SQLAlchemy async session management challenges.
- Docker Compose (5 services) has higher initial setup — accepted because it provides consistency, reproducibility, and mirrors production from Day 1.

13. Rejected Options & Rationale

Option 1: Node.js (Express) + Next.js + MongoDB — REJECTED

- Rejected due to a fundamental data model mismatch and ecosystem gaps:
- MongoDB's document model cannot enforce referential integrity (foreign keys) between patients, appointments, slots, and doctors at the database level. For a scheduling system where data consistency is business-critical (a booking MUST reference a valid patient and a valid slot), relying on application-level enforcement is unacceptable.
- Express.js provides minimal structure — no built-in serialization, validation framework, permission system, or admin panel. Every capability that Django DRF provides out of the box must be manually assembled (Joi/Zod, Passport.js, custom middleware), increasing development time and bug surface.
- Node.js is weaker than Python for AI/voice workloads. Whisper cannot run locally in Node.js (requires Python subprocess). The Python Twilio SDK is more feature-complete. The AI/ML ecosystem heavily favors Python.
- No admin panel equivalent — Django admin provides instant data inspection and management for free. Express requires building custom admin UI or integrating AdminJS, adding 1–2 weeks.
- The unified TypeScript advantage does not compensate for the cumulative weaknesses in data integrity, AI ecosystem, and tooling maturity.

Option 3: Python FastAPI + Next.js + PostgreSQL — REJECTED

- Rejected as a strong but slower alternative to Option 2:
- FastAPI's lack of a built-in admin panel is the primary disadvantage. Django admin saves 1–2 weeks of development and provides invaluable prototype demo capability. FastAPI requires building or integrating SQLAdmin, which is less mature.
- SQLAlchemy + Alembic has a steeper learning curve and more verbose syntax than Django ORM — model definitions, migration generation, and session management all require more boilerplate code.

- FastAPI provides no project structure convention — the team must design folder layout, dependency injection patterns, and service layers from scratch. Django’s app-based convention provides this for free.
- ARQ is less proven than Celery for production task processing — smaller community, no Flower-equivalent monitoring, less documentation. Using Celery with FastAPI is possible but negates FastAPI’s lightweight advantage.
- FastAPI’s raw performance advantage (async-first) is irrelevant at MediSched’s scale (20 concurrent users, 100 calls/day). Django’s performance is more than adequate.
- While FastAPI shares Python’s AI ecosystem advantage, the cumulative productivity gap vs. Django DRF (admin panel + ORM + migrations + DRF viewsets) makes it the slower path to a 4-week prototype.

14. Key Risks & Mitigations

The following risks are specific to the recommended Option 2 and must be actively managed:

ID	Risk	Likelihood	Mitigation
CR-01	Celery worker crash during active call orchestration	High	Celery auto-retry with exponential backoff (max 3 retries). Redis-backed task persistence ensures jobs survive worker restarts. Docker restart: always policy. Idempotent task design — re-processing a call attempt checks current state before acting.
CR-02	Django Channels WebSocket drops under 20 concurrent users	Low	Redis channel layer (not in-memory) is production-grade for this scale. ASGI server (Daphne/Uvicorn) handles WebSocket natively. Client-side auto-reconnect with exponential backoff. Load test at Week 3.
CR-03	API contract drift between Django	Medium	DRF auto-generates OpenAPI schema

	backend and React frontend		(drf-spectacular). React uses generated TypeScript types. API contract validated in CI pipeline.
CR-04	Whisper API latency delays call log availability	Medium	Transcription is fully asynchronous via Celery. Call logs show "Transcription in progress" status until worker completes. No blocking dependency.
CR-05	Twilio free-tier rate limits hit during demo	High	Plan test scenarios within free-tier limits. Budget \$20–\$50 for paid tier during demo week. Mock Twilio in unit/integration tests using responses library.
CR-06	Docker Compose environment inconsistencies	Low	.env.example template committed to repo. Docker Compose pins all image versions. CI pipeline validates docker-compose build on every push. README documents setup.
CR-07	RBAC implementation skipped under prototype time pressure	Medium	DRF permission classes implemented in Sprint 1 as foundational middleware. RBAC is in the critical path — all viewsets require explicit permission_classes. No endpoint ships without RBAC.
CR-08	Django ORM N+1 queries degrade calendar performance	Medium	Use select_related() and prefetch_related() on all queryset-heavy

			endpoints. Django Debug Toolbar in development to catch N+1 issues early. NFR-P-01 (2s page load) validated in load test.
--	--	--	---

15. Open Questions

The following questions must be resolved during PRISM-S (Spec & Design) before implementation begins.

OQ-I-01: Mono-repo or separate repos for Django backend and React frontend?

Owner: Engineering Lead. Recommendation: Mono-repo with /backend and /frontend directories. Simpler CI/CD, shared Docker Compose, and easier onboarding for a small team.

OQ-I-02: Local Whisper model vs. Whisper API for the prototype?

Owner: Engineering Lead. Recommendation: Whisper API for prototype (simpler, no GPU dependency). Evaluate local model for production if transcription volume exceeds API cost threshold.

OQ-I-03: React state management — React Query vs. Redux Toolkit?

Owner: Frontend Developer. Recommendation: React Query (TanStack Query) for server state (API data fetching, caching, refetching). Local UI state via React useState/useContext. Redux only if complex client-side state emerges.

OQ-I-04: Django Channels ASGI server — Daphne vs. Uvicorn?

Owner: Engineering Lead. Recommendation: Uvicorn (faster, actively maintained, async-native). Daphne as fallback if Channels compatibility issues arise.

OQ-I-05: Should Celery Flower (monitoring dashboard) be included in the prototype Docker Compose?

Owner: Engineering Lead. Recommendation: Yes — adds one container but provides invaluable task visibility during development and demo. Accessible only to admin.

OQ-I-06: What is the sprint plan for the 8-week timeline?

Owner: Product Owner + Engineering Lead. Proposed plan:

Sprint 1 (Weeks 1–2): Django project setup, Docker Compose, models for all entities, DRF serializers/viewsets for Patient and Doctor management, JWT auth + RBAC, React project setup with routing and auth.

Sprint 2 (Weeks 3–4): Doctor slot management (release, block, auto-generate), Calendar view (FullCalendar + WebSocket), appointment booking flow, Django admin configuration.

PROTOTYPE DEMO.

Sprint 3 (Weeks 5–6): Celery setup, AI outbound calling (Twilio integration, 3-attempt escalation), call transcription (Whisper), call log management.

Sprint 4 (Weeks 7–8): Automated reminders (SMS/email), analytics dashboard + CSV export, edge case handling, security hardening, load testing, final demo.

16. Decision & Approval

Selected Solution: Option 2 – Django REST Framework + React.js + PostgreSQL + Celery + Redis

Why this stack over the alternatives:

Evaluated Stack	Verdict	Key Differentiator
Express + Next.js + MongoDB	Rejected	MongoDB lacks relational integrity for scheduling data. No admin panel. Node.js is weaker for AI/voice.
Django DRF + React.js + PostgreSQL	Selected (28/30)	Batteries-included: admin panel, ORM, DRF, Celery, built-in security. Best productivity-to-quality ratio.
FastAPI + Next.js + PostgreSQL	Rejected	Strong but slower to prototype. No admin panel. SQLAlchemy/Alembic more verbose. ARQ less mature than Celery.

Final Technology Stack:

Layer	Technology	Purpose
Frontend	React.js (Vite)	SPA for scheduling dashboard, patient/doctor management, analytics
Backend API	Django 5.x + Django REST Framework	REST API, business logic, validation, RBAC, ORM
Database	PostgreSQL	Relational data store with ACID compliance and referential integrity
Real-Time	Django Channels + Redis	WebSocket-based live calendar updates

Background Jobs	Celery + Redis (broker)	AI call orchestration, transcription, reminders
Task Monitoring	Celery Flower	Real-time task queue monitoring dashboard
Voice/Telephony	Twilio Python SDK	Outbound AI calls, TwiML IVR, SMS reminders
Speech-to-Text	OpenAI Whisper API	Call audio transcription
Email	Django send_mail / SendGrid	Email reminders
Auth	Simple JWT + DRF Permissions	JWT tokens with role-based access control
Containerization	Docker Compose	5 services: react, django, celery, postgres, redis

Decision Owner Approval:

Name: _____

Signature / Approval: _____

Date: _____

By signing above, the Decision Owner confirms that:

- All three solution options have been reviewed and evaluated.
- Option 2 (Django REST Framework + React.js + Celery) is approved as the solution architecture.
- Rejected options and their rationale are acknowledged.
- Key risks are documented with mitigations.
- Open questions have assigned owners for resolution in PRISM-S.

17. AI Gold Prompt Mapping (Mandatory)

This section defines the approved AI prompt structure to be used when generating solution options and tradeoff analysis. Free-form prompting is not allowed for PRISM-I.

17.1 Approved AI Tool(s)

- Primary: ChatGPT
- Secondary (feasibility spikes / repo checks): Claude Code

17.2 Gold Prompt – PRISM-I

ROLE:

You are a Senior System Design Architect at SmartX Technologies.

OBJECTIVE:

Propose and compare multiple viable solution approaches for MediSched AI.

CONTEXT:

- Company: SmartX Technologies
- Project type: Web Application (SaaS Platform) — Healthcare Diagnostic Center
- Technology pool from project documentation: Python (FastAPI/Django), Node.js (Express), React.js, Next.js, PostgreSQL, MongoDB
- NFRs ranked: Time to Deliver (Critical), Functional Fit (High), NFR Alignment (High), Complexity (High), Cost (Medium), Operational Risk (Medium)
- Constraints: 8-week timeline, 4-week prototype, 2–3 developers, free-tier services, RBAC + encryption mandatory

INPUTS:

- PRISM-R PRD-Lite: 50 FRs across 9 modules, 17 NFRs, 12 edge cases, 4 user roles, 3 system actors, 4 external integrations
- PRISM-P resolved decisions: PostgreSQL, English-only, simulated pilot, 3-attempt call escalation, admin slot fallback, CSV export only
- No existing system — greenfield build

INSTRUCTIONS:

- Propose at least 3 genuinely different solution options (including minimal change)
- Do NOT converge early; explore tradeoffs
- Evaluate each option against functional and NFR criteria
- Identify risks, complexity, and operational impact

OUTPUT FORMAT:

- 1. Solution Options Overview
- 2. Detailed description per option
- 3. Pros / Cons / Risks per option
- 4. Tradeoff Comparison Matrix
- 5. Recommended Option with rationale

QUALITY CHECK:

- Are there at least 3 viable options?
- Are tradeoffs explicit and tied to NFRs?
- Is the recommendation clearly justified?

17.3 Evidence for PRISM Gate (M2 → M3)

- Completed Solution Options Matrix
- Minimum three options evaluated
- Tradeoff comparison completed
- Decision owner approval
- Updated risks and open questions